

Web Application Security Assessment Report

SQL Injection Vulnerability Allowing Authentication (Login) Bypass

Prepared by Mohammed Ashraf Mokhtar

Prepared By	Mohammed Ashraf Mokhtar
LinkedIn	linkedin.com/in/1mohammed-ashraf
GitHub	github.com/moashraf2006
Target Application	we-like-to-shop.net (PortSwigger Web Security Academy training environment)
Host	0a96000f04c70d9d82ebd8aa00e00063.web-security-academy.net
Assessment Type	Manual black-box penetration test — SQL Injection
Vulnerable Endpoint	POST /login (username parameter)
Tooling	Burp Suite Community Edition v2026.7.3 (browser-based testing)
Classification	CWE-89: Improper Neutralization of Special Elements used in an SQL Command
Severity	Critical
Status	Confirmed — Exploited Successfully — Lab Solved
Date of Testing	2 September 2026

This document is a technical write-up of a controlled exercise performed against a PortSwigger Web Security Academy training lab. All testing was conducted against an intentionally vulnerable, isolated instance provided for educational purposes. No production systems or real user data were involved.

1. Executive Summary

During testing of the target application's authentication mechanism, a SQL injection vulnerability was identified in the **username** field of the `/login` endpoint. The application concatenates the submitted username directly into a server-side SQL query used to verify credentials, without sanitisation or parameterisation.

By submitting a username payload that comments out the password check entirely, authentication was bypassed and a session was established as the **administrator** user without knowledge of the real credentials. This confirms that an unauthenticated attacker can fully impersonate any known user account, including privileged accounts, satisfying the lab's success condition.

Impact: Complete authentication bypass via unauthenticated SQL injection, allowing an attacker to log in as any user — including administrator — without valid credentials. This grants full access to that user's account and any privileged functionality available to it.

CRITICAL

Key Findings

- The username POST parameter is inserted into a SQL query without input validation or use of parameterised queries.
- A single quote character breaks out of the intended string literal, enabling injection of arbitrary SQL logic.
- The payload `administrator'--` terminates the username literal and comments out the remainder of the query, including the password comparison.
- The password field value is not evaluated at all once the comment marker is injected, so any value in that field is accepted.
- The vulnerability was confirmed and successfully exploited via the application's own login form; the resulting session was authenticated as **administrator** and the lab was marked **Solved**.

2. Scope & Methodology

2.1 Scope

Testing was limited to the authentication functionality of the single-page 'We Like To Shop' e-commerce demo application hosted on the PortSwigger Web Security Academy platform, specifically the login form reachable at /login.

2.2 Methodology

The login form was tested directly through the browser, submitting boundary-testing and injection payloads into the username field to probe how the backend authentication query handles unsanitised input. The standard black-box SQL injection workflow for authentication bypass was followed:

- 1 Identify a login form that authenticates users against a backend data store, a classic candidate for SQL injection.
- 2 Establish baseline behaviour by submitting a normal (invalid) username and password and observing the failure response.
- 3 Submit a single quote in the username field to check for a change in behaviour, confirming the input reaches a SQL query unsanitised.
- 4 Construct a payload that closes the username string literal and comments out the remainder of the query — including the password check — using a known/likely-valid username such as administrator.
- 5 Submit the crafted request with an arbitrary password value and verify whether authentication succeeds.
- 6 Confirm exploitation success by checking the resulting session's identity and the lab's defined objective.

2.3 Tools Used

Interception Proxy	Burp Suite Community Edition v2026.7.3 (Chromium-embedded browser)
Testing Method	Direct form submission via the application's login page
Browser	Chromium-based browser (Burp-embedded), Windows 10 / Edge UA

3. Vulnerability Details

Vulnerability	SQL Injection — Authentication Bypass (comment-based)
CWE Reference	CWE-89 — Improper Neutralization of Special Elements used in an SQL Command
CWE (secondary)	CWE-287 — Improper Authentication
OWASP Category	A03:2021 — Injection / A07:2021 — Identification and Authentication Failures
Affected Parameter	username (POST, /login endpoint)
Authentication Required	None — publicly reachable, unauthenticated
CVSS 3.1 (indicative)	9.8 (Critical) — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

3.1 Root Cause

When a user submits the login form, the application builds and executes a server-side SQL statement by directly concatenating the unsanitised username and password values into the query string, in a form equivalent to:

SQL — server-side authentication query (reconstructed)

```
SELECT * FROM users
WHERE username = 'wiener' AND password = 'peter'
```

Because the parameter values are placed inside string literals without escaping or parameter binding, a single quote followed by a SQL comment marker in the username field terminates the literal early and comments out everything that follows — including the entire password check.

3.2 Proof of Concept — Evidence Log

Step 1 — Lab reconnaissance. The lab brief confirms the objective: perform a SQL injection attack that logs in to the application as the administrator user.

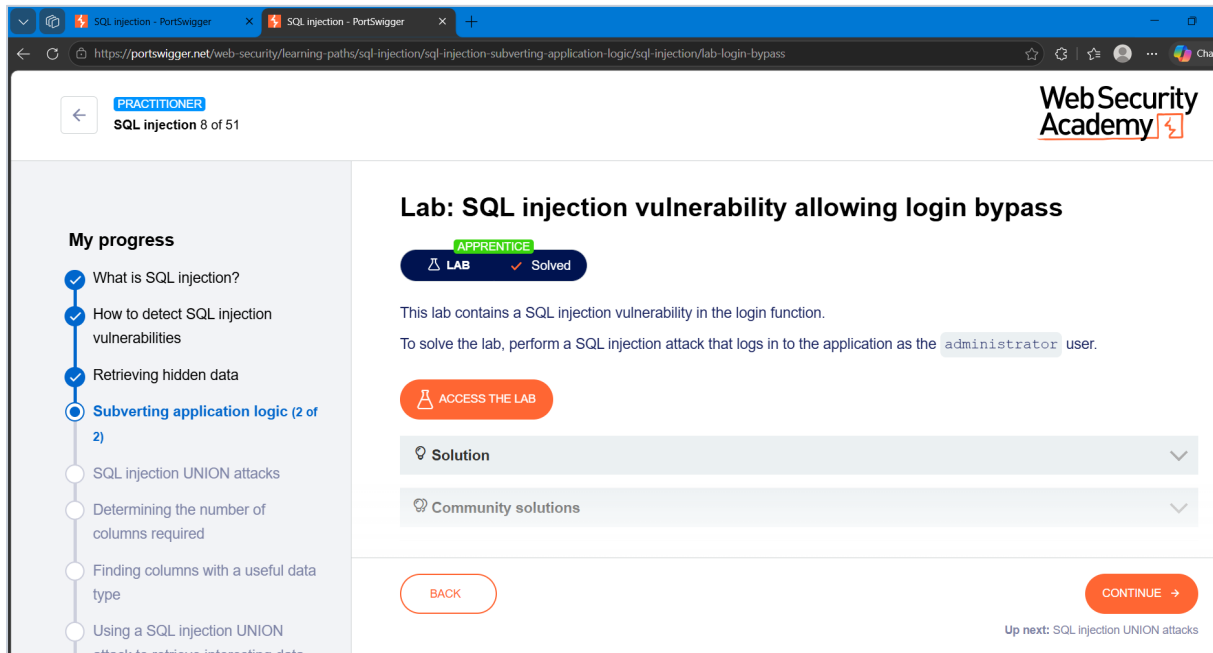


Figure 1. PortSwigger lab brief for “SQL injection vulnerability allowing login bypass,” stating the objective of logging in as administrator.

Step 2 — Baseline reconnaissance. Browsing the target’s “We Like To Shop” storefront confirms the lab is not yet solved and that the application exposes a standard Home | My account navigation, with authentication handled via a login form.

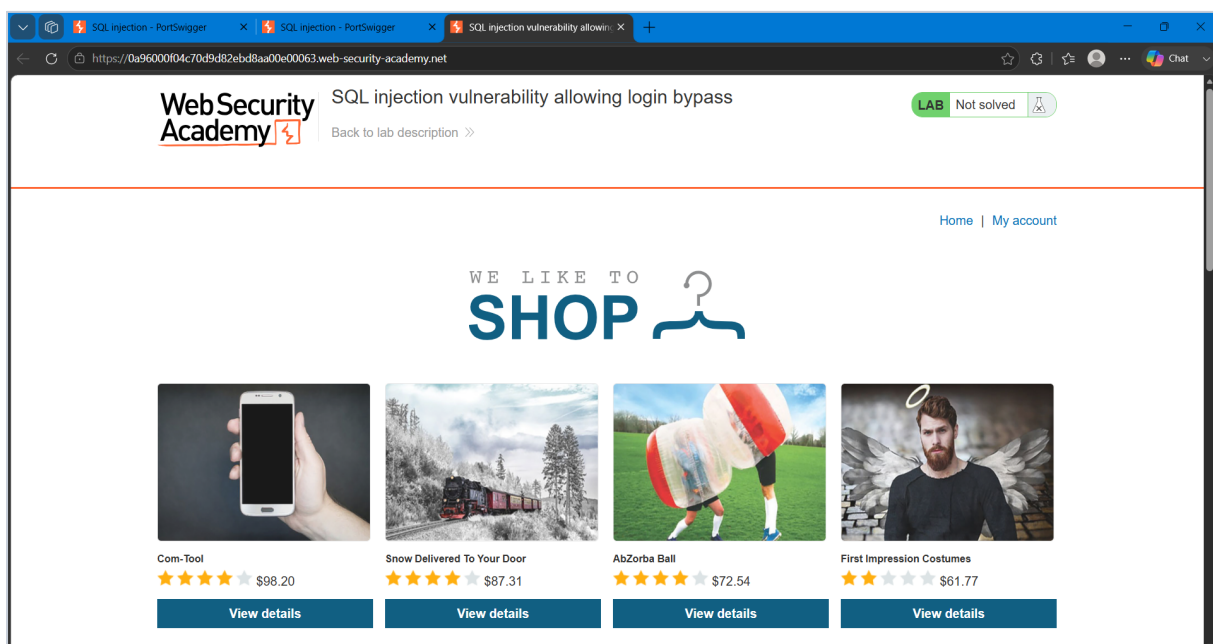


Figure 2. Target application in its default state — LAB status “Not solved.”

Step 3 — Locate the login form. Navigating to /login exposes a standard username/password authentication form — the injection point under test.

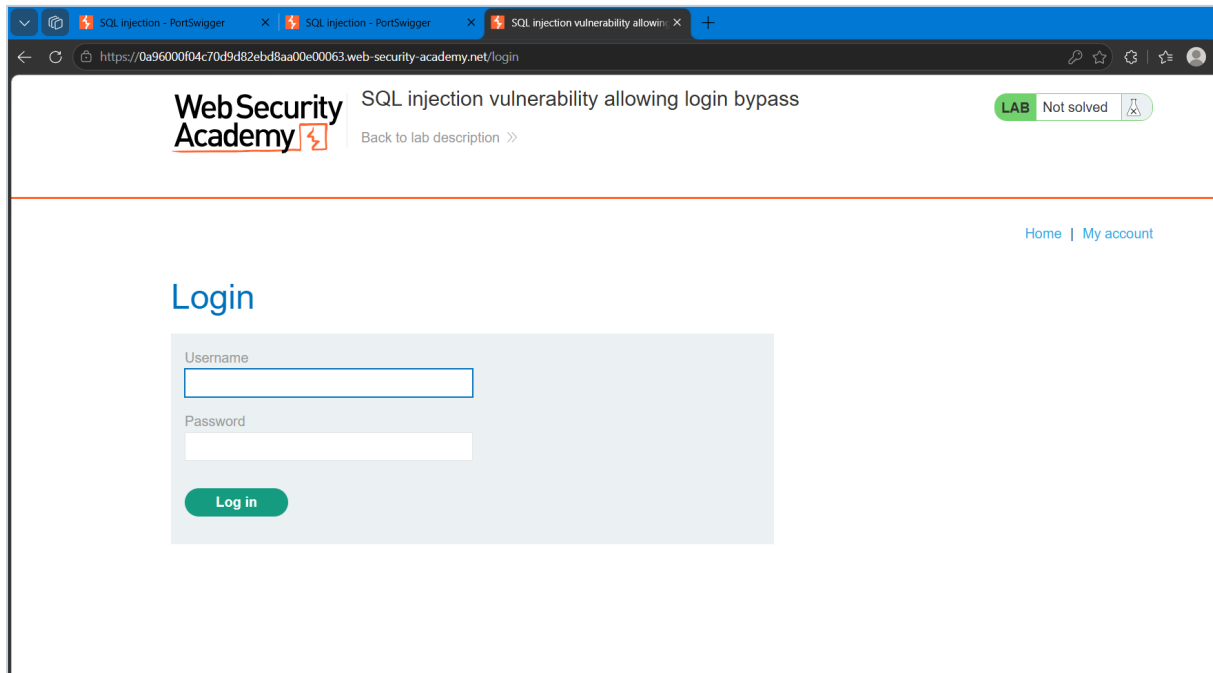


Figure 3. The application's login page prior to any input, LAB status still "Not solved."

Step 4 — Injection payload and submission. The username field was set to a payload designed to close the string literal and comment out the remainder of the query, using the known privileged account name. The password field was left with an arbitrary value, since it is never evaluated once the query is truncated by the comment marker:

Payload used

Username: administrator'--

Password:

Resulting query interpreted by the database:

SQL — effective query after injection

```
SELECT * FROM users
```

```
WHERE username = 'administrator'--' AND password = ''
```

The trailing double-dash (--) comments out the remainder of the statement, including the entire `AND password = ''` clause. The database therefore only evaluates `username = 'administrator'`, which is true for the administrator account regardless of whatever password value was submitted.

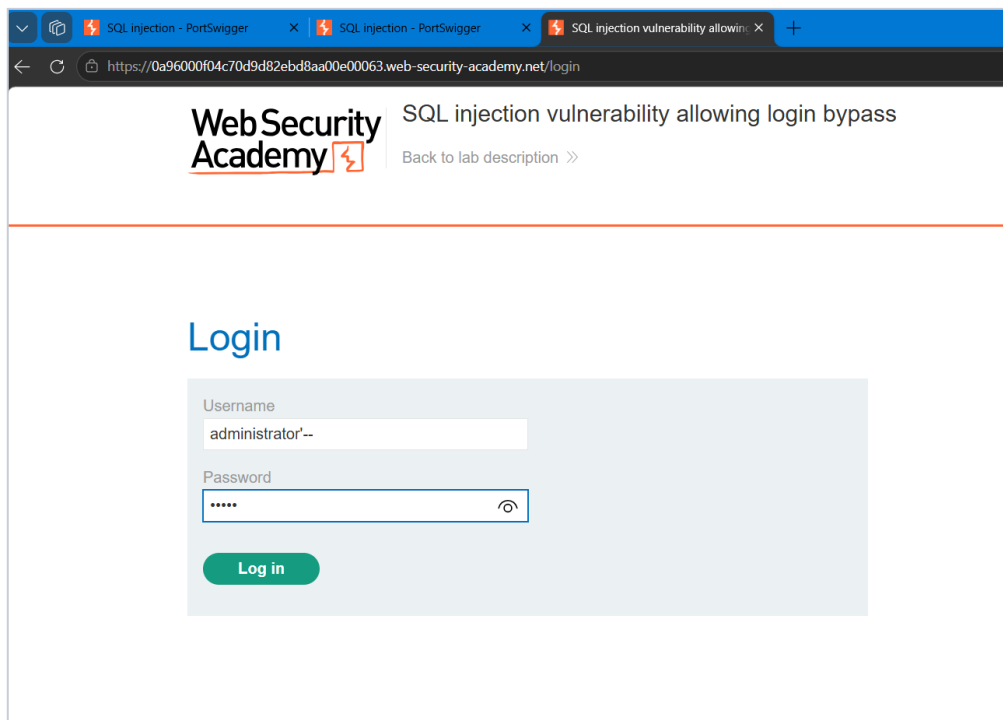


Figure 4. Login form submitted with username administrator'-- and an arbitrary password.

Step 5 — Result. The application authenticated the request as the administrator user and redirected to the account page, which resolves to `/my-account?id=administrator` and confirms the identity of the logged-in session. The LAB status flips to **Solved**, with the application's own success banner confirming exploitation.

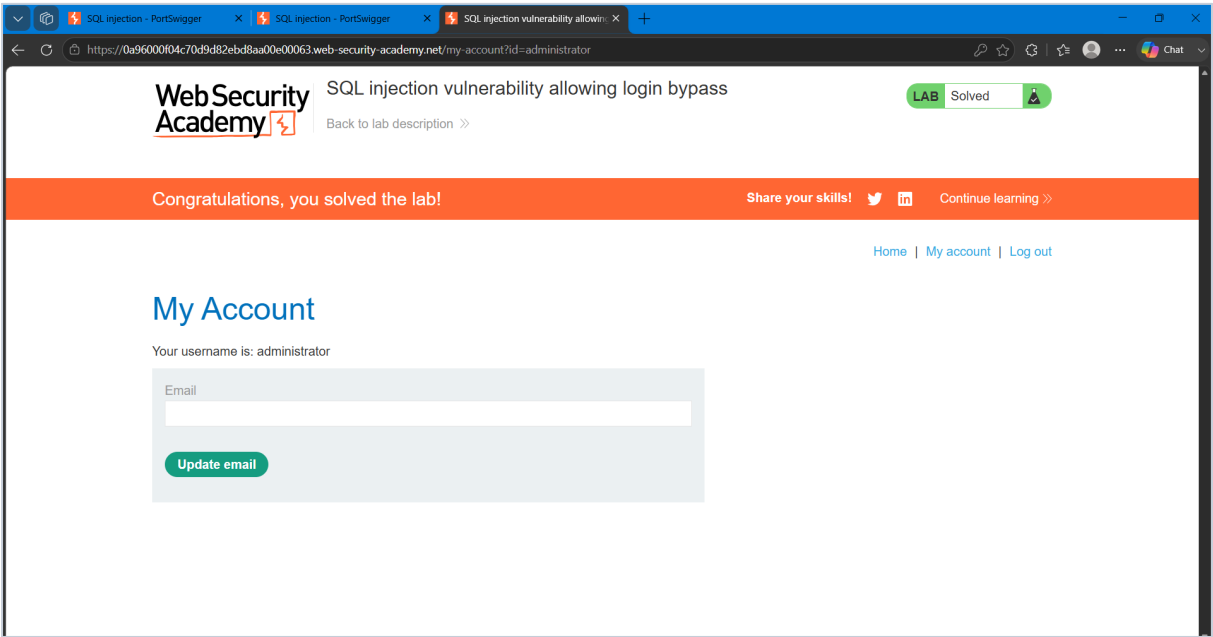


Figure 5. “My Account” page reached as the administrator user — “Your username is: administrator,” LAB status “Solved,” and the application’s “Congratulations, you solved the lab!” banner confirming successful exploitation.

Result Summary

HTTP Response	302 redirect → 200 OK on /my-account?id=administrator
Observed Effect	Password check bypassed entirely; session authenticated as the administrator user without valid credentials
Lab Status	Solved — success banner returned: “Congratulations, you solved the lab!”

4. Impact Assessment

This vulnerability allows a complete bypass of the authentication mechanism for any account whose username is known or guessable, with no need to know — or even guess — the corresponding password. Because the administrator account was successfully targeted, the practical impact in a real-world deployment of this pattern would typically include:

- Full takeover of any user account, including administrative accounts, without valid credentials.
- Access to privileged functionality normally restricted to administrators (user management, configuration, sensitive data).
- Exposure of personal or business data belonging to the impersonated account.
- A foothold for further attacks, since an authenticated administrative session often unlocks additional attack surface (e.g. stored SQL injection in other authenticated features, privilege-dependent functionality).

Because the login endpoint requires no prior authentication and the attack requires no user interaction beyond submitting the form, this is trivially exploitable by any anonymous visitor who knows or guesses a valid username — which further increases its severity.

5. Remediation

The following corrective actions are recommended, in priority order:

- 1 **Use parameterised queries / prepared statements** for all authentication logic, so that user input is always treated as data and never as executable SQL syntax. Example (illustrative): `SELECT * FROM users WHERE username = ? AND password = ?`, with both values bound as parameters.
- 2 **Never build authentication queries via string concatenation**, even for seemingly low-risk fields — the username field in this case was sufficient on its own to bypass the password check entirely.
- 3 **Store and compare password hashes** using a strong, salted hashing algorithm (e.g. bcrypt, Argon2) rather than comparing plaintext values in a SQL query, as defence in depth.
- 4 **Adopt an ORM or query-builder** that parameterises input by default, reducing the risk of ad-hoc string concatenation reappearing elsewhere in the codebase.
- 5 **Apply strict server-side input validation** on the username field (e.g. character allow-listing) as a defence-in-depth measure, in addition to parameterisation — never as a substitute for it.
- 6 **Implement account lockout / rate limiting** on the login endpoint to slow down automated exploitation attempts and credential/injection probing.
- 7 **Conduct a codebase-wide review** for the same anti-pattern, since a single instance of unsanitised query construction in authentication logic often indicates the pattern exists elsewhere in the application.

6. Conclusion

This assessment confirmed a Critical-severity SQL injection vulnerability in the login functionality of the target application. The vulnerability was successfully exploited using a comment-based payload in the username field, resulting in a full authentication bypass and access to the administrator account without valid credentials. The lab objective was met and the exercise was marked as **Solved** by the platform.

Remediation should prioritise migrating all authentication queries to parameterised statements and moving password verification to a proper hashing scheme, which together eliminate this class of vulnerability at the root cause rather than relying solely on input filtering.

Reference

PortSwigger Web Security Academy — “SQL injection vulnerability allowing login bypass”, Subverting application logic (2 of 2), SQL injection module, lab 8 of 51.